

1. Downstream task

A downstream task olyan specifikus feladat, amelyre egy általánosan betanított (pretrained) modellt finomhangolunk vagy értékelünk.

A GraphCodeBERT (mint pre-trained model) kód-reprezentációkat tanul, de nem konkrét feladatra. A downstream task az, ahol ezeket a reprezentációkat alkalmazzuk egy célra.

A leggyakoribb downstream taskok kódmodelleknél:

2. Code Search Task

Cél:

Képesnek lenni természetes nyelvű lekérdezést (query, pl. "sort a list of numbers") és kódrészleteket közös vektortérben reprezentálni, hogy a két típus közti hasonlóságot (cosine similarity) mérni lehessen.

A legközelebbi embedding a leginkább releváns kód.

Input formátum:

- Natural Language (NL): rövid leírás vagy kérdés
- Code Snippet: a megfelelő kódrészlet

Training Objective (Loss Function):

GraphCodeBERT az ún. contrastive learning elvet használja:

- cél, hogy az igaz (NL, Code) pár embeddingje közel legyen egymáshoz a vektortérben,
- míg a nem kapcsolódó párok embeddingje távol legyen.

3. Hogyan valósította meg ezt az eredeti GraphCodeBERT

A [GraphCodeBERT paper](#) a CodeSearchNet benchmarkon validálta a modellt.

A CodeSearchNet több nyelven (Python, Go, Java, JS, PHP, Ruby) tartalmaz <natural language, code> párokat.

3.1 Architektúra

- Egy shared GraphCodeBERT encoder dolgozza fel mind az NL, mind a Code inputot.
- A két embeddinget egy 768-dimenziós vektortérbe képezi.
- A CLS token embeddingjét használja reprezentációként mindkét inputhoz.

- A training során dual-encoder stílusban (két forward pass) fut
- A cosine similarity alapján ranking loss-t számít.

3.2 Data pipeline

1. Input párok betöltése (docstring + code)
2. Tokenizálás mindkettőre
3. Embeddingek kinyerése
4. Contrastive training
5. Értékelés:
 - Egy NL query-hez kiszámítja az összes kód embeddinget.
 - Megnézi, hogy az eredeti pár hanyadik helyen van a cosine similarity szerint (MRR@k, Recall@k).

4. Code Search megvalósítása

Dataset előkészítése

Olyan adathalmazt kell létrehoznunk, amely nem csak kódot, hanem NL + kód párokat tartalmaz. Az ErlangBERT korábbi fázisaiban már előállítottuk a functions.jsonl fájlt, amely soronként tartalmazott Erlang-függvényeket a következő formában:

```
{
  "idx": "happi-theBeamBook::compile/2::0",
  "repo": "happi-theBeamBook",
  "path": "code\\beam_chapter\\src\\stack_machine_compiler.erl",
  "func_name": "compile",
  "code": "compile(Expression, FileName) -> file:write_file(FileName, generate_code(ParseTree) ++ [stop()]).",
  "docstring": "",
  "language": "erlang"
}
```

Ez az adatformátum csak a kódot tartalmazta, viszont a code search feladat megköveteli, hogy minden bejegyzéshez tartozzon egy természetes nyelvű leírás (NL) is, illetve fenn kellett tartani, hogy megfeleljen a CodeSearchNet formátumával.

Ehhez egy scriptet használtam, amely heurisztikusan készített leírást a kódokhoz. Ez az első működő verzióhoz megfelel, de ennél sokkal pontosabb NL leírásra lesz majd szükség.

A script a functions.jsonl adatait felhasználva olyan bejegyzéseket hozott létre, ahol az NL rész automatikusan épült a függvénynévből, az aritásból, a fájlból és a mappanévből:

```
{ "nl": "erlang function tolower s; arity 1; src, misc; pattern matching",  
  "code": "tolower_s([C | Cs]) -> if C >= $A, C =< $Z -> [C + 32 | tolower_s(Cs)]; true -> [C | tolower_s(Cs)] end;  
  tolower_s([]) -> [].",  
  "repo": "processone-ejabberd",  
  "func_name": "tolower_s",  
  "lang": "erlang",  
  "split": "train" }
```

Input tokenizálás

- Mind az NL, mind a Code stringet külön tokenize-oljuk
- A tokenizáláshoz a GraphCodeBERT-hez tartozó RoBERTa tokenizer-t használtam:

```
from transformers import RobertaTokenizer  
tokenizer = RobertaTokenizer.from_pretrained("microsoft/graphcodebert-base")  
inputs_nl = tokenizer(docstring, truncation=True, max_length=128, return_tensors="pt")  
inputs_code = tokenizer(code, truncation=True, max_length=256, return_tensors="pt")
```

Embeddingek kinyerése

- A GraphCodeBERT-alapú modell (Erlangra finomhangolva) képes a szöveg és a kód embeddingjeit (vektoros reprezentációit) előállítani.
- A CLS token szolgált az egész bemenet reprezentációjaként:

```
from transformers import RobertaModel  
model = RobertaModel.from_pretrained("models/erlangbert_full_finetune_v2/checkpoint-best")  
h_nl = model(**inputs_nl).last_hidden_state[:, 0, :]  
h_code = model(**inputs_code).last_hidden_state[:, 0, :]
```

Hasonlóság számítása

- A kód és az NL embeddingek között cosine similarity-t használtam:
- Ez a hasonlósági érték megmutatja, hogy az adott NL leírás és a kód mennyire közel helyezkedik el a vektortérben.
- A cél az, hogy a helyes párok hasonlósága magas legyen, a hibásaké pedig alacsony.

```
import torch  
similarity = torch.nn.functional.cosine_similarity(h_nl, h_code)
```

Training

- A tanítás során a kontrasztív tanulás elvét alkalmaztam.
- Minden batch több (NL, Code) párt tartalmaz, és a veszteségfüggvény (contrastive loss) azt próbálja elérni, hogy:
 - a helyes NL–Code párok embeddingjei közel legyenek egymáshoz,
 - a nem párosított példák embeddingjei távol legyenek egymástól.

- `loss = contrastive_loss(h_nl, h_code)`

```
python train\train_codesearch.py `
--base-checkpoint models\erlangbert_full_finetune_v2\checkpoint-best `
--train-file output\code_search_data\train.jsonl `
--valid-file output\code_search_data\valid.jsonl `
--out-dir models\erlang_graphcodebert_codesearch `
--batch-size 32 `
--epochs 4 `
--lr 2e-5 `
--temperature 0.07
```

Elakadás és megoldás

A model.bin nem volt HuggingFace formátumú checkpoint

A from_pretrained() metódus pytorch_model.bin-t keresett, de a saját checkpointomban csak model.bin volt. Ezt egy konverziós scripttel oldottam meg, amely ezt az átalakítást elvégezte.

Értékelés

Az értékeléshez készítettem egy külön eval/eval_code_search.py scriptet, amely:

- betölti a tanított modellt,
- beolvassa a test.jsonl adathalmazt,
- előállítja a kód és a lekérdezés embeddingeket,
- majd kiszámolja a MRR és Recall@K metrikákat.

```
python eval\eval_code_search.py `
--model-checkpoint models\erlang_graphcodebert_codesearch\checkpoint-best `
--data-file output\code_search_data\test.jsonl `
--batch-size 64 `
--k 1 3 5 10 `
--save-report eval\codesearch_report.json `
--save-embeddings output\code_search_index\test_code_embeds.npz
```

Eredmény:

```
=== Metrics ===
MRR: 0.7422
Recall@1: 0.6792
Recall@3: 0.7856
Recall@5: 0.8267
Recall@10: 0.8772
```

5. Összegzés

Ezzel a pipeline-nal sikerült az ErlangBERT-et kiterjesztenem egy funkcionális code search modellé.

A modell már képes:

- természetes nyelvű kérdés alapján Erlang-függvényeket rangsorolni,
- a releváns kódrészleteket az embeddingtérben megtalálni.

A következő lépés a pipeline-ban a:

- interaktív kereső megírása (query $\rightarrow$ top-5 kód),
- valamint az NL-oldal továbbfejlesztése gazdagabb leírásokkal, hogy még pontosabb eredményeket adjon.